



Aula 06 - Tailwind CSS

▼ Tipo	Aula
▼ Bimestre	1

Design Responsivo

Usando variantes de utilitários responsivos para construir interfaces de usuário adaptáveis.

Visão geral

Cada classe de utilitário no Tailwind pode ser aplicada condicionalmente em **diferentes pontos de interrupção**, o que torna muito fácil construir interfaces responsivas complexas sem nunca sair do HTML.

Existem cinco pontos de interrupção por padrão, inspirados em resoluções comuns de dispositivos:

Prefixo do ponto de interrupção	Largura mínima	CSS
sm	640 px	@media (min-width: 640px) { ... }
md	768px	@media (min-width: 768px) { ... }
lg	1024px	@media (min-width: 1024px) { ... }
xl	1280 px	@media (min-width: 1280px) { ... }
2xl	1536px	@media (min-width: 1536px) { ... }

Para adicionar um utilitário, mas apenas fazer com que ele entre em vigor em um determinado ponto de interrupção, tudo o que você precisa fazer é prefixar o utilitário com o nome do ponto de interrupção, seguido do `:` caractere:

```
<!-- Width of 16 by default, 32 on medium screens, and 48 on large screens -->

```

Isso funciona para **todas as classes de utilitários da estrutura**, o que significa que você pode alterar literalmente qualquer coisa em um determinado ponto de interrupção — até mesmo coisas como espaçamento entre letras ou estilos de cursor.

Working mobile-first

Por padrão, o Tailwind usa um **sistema de breakpoint mobile-first**, semelhante ao que você pode estar acostumado em outras estruturas como Bootstrap.

O que isso significa é que utilitários não prefixados (como `uppercase`) têm efeito em todos os tamanhos de tela, enquanto utilitários prefixados (como `md:uppercase`) só têm efeito no ponto de interrupção especificado e *acima*.

Segmentação de telas de dispositivos móveis

O que mais surpreende as pessoas nessa abordagem é que, para estilizar algo para dispositivos móveis, você precisa usar a versão sem prefixo de um utilitário, não a `sm:` versão com prefixo. Não pense nisso `sm:` como significando “em telas pequenas”, pense nisso como “no pequeno *ponto de interrupção*”.

Não use `sm:` para segmentar dispositivos móveis

```
<!-- Isso centralizará o texto apenas em telas de 640px e maiores, e não em telas pequenas -->
<div class="sm:text-center"></div>
```

Use utilitários sem prefixo para direcionar dispositivos móveis e substitua-os em pontos de interrupção maiores

```
<!-- Isso centralizará o texto no celular e o alinhará à esquerda em telas de 640px e maiores -->
<div class="text-center sm:text-left"></div>
```

Por esse motivo, geralmente é uma boa ideia implementar primeiro o layout móvel para um design e, em seguida, aplicar quaisquer alterações que façam sentido para as telas `sm`, seguidas pelas telas `md`, etc.

Container

Um componente para fixar a largura de um elemento ao ponto de interrupção atual.

Class	Breakpoint	Properties
<code>container</code>	<i>None</i>	<code>width: 100%;</code>
	<i>sm (640px)</i>	<code>max-width: 640px;</code>
	<i>md (768px)</i>	<code>max-width: 768px;</code>
	<i>lg (1024px)</i>	<code>max-width: 1024px;</code>
	<i>xl (1280px)</i>	<code>max-width: 1280px;</code>
	<i>2xl (1536px)</i>	<code>max-width: 1536px;</code>

Usando o contêiner

A classe `container` define o valor `max-width` de um elemento para corresponder ao valor `min-width` do ponto de interrupção atual. Isso é útil se você preferir projetar para um conjunto fixo de tamanhos de tela em vez de tentar acomodar uma janela de visualização totalmente fluida.

Observe que, diferentemente dos contêineres que você pode ter usado em outras estruturas, **o contêiner do Tailwind não se centraliza automaticamente e não possui nenhum preenchimento horizontal integrado.**

Para centralizar um contêiner, use o `mx-auto` utilitário:

```
<div class="container mx-auto">
  <!-- ... -->
</div>
```

Para adicionar preenchimento horizontal, use os `px-{size}` utilitários:

```
<div class="container mx-auto px-4">
  <!-- ... -->
</div>
```

Variantes responsivas

A classe `container` também inclui variantes responsivas, como `md:container` por padrão, que permitem fazer algo se comportar como um contêiner apenas em um determinado ponto de interrupção e acima:

```
<div class="md:container md:mx-auto">
  <!-- ... -->
</div>
```

Grid Template Columns

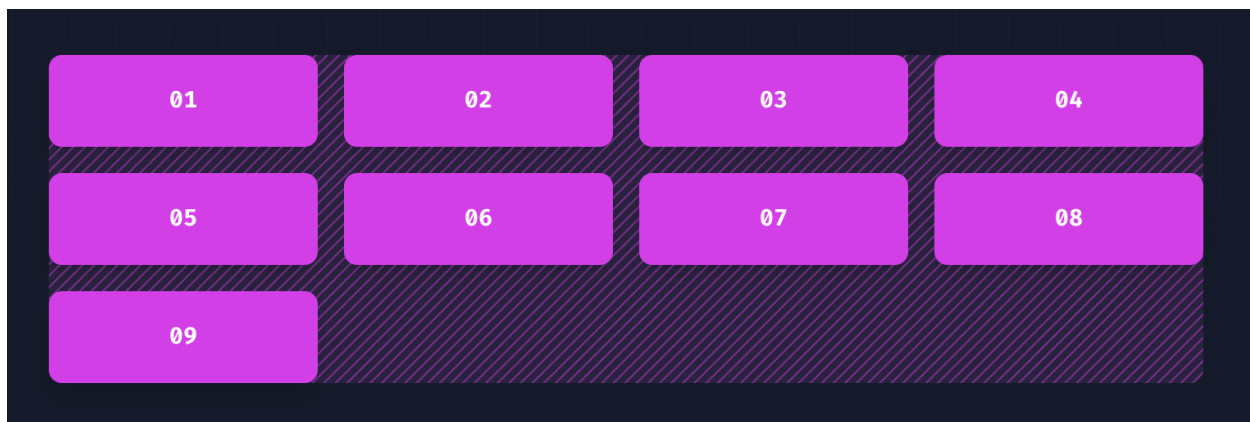
Utilitários para especificar as colunas em um layout de grade.

Class	Properties
grid-cols-1	grid-template-columns: repeat(1, minmax(0, 1fr));
grid-cols-2	grid-template-columns: repeat(2, minmax(0, 1fr));
grid-cols-3	grid-template-columns: repeat(3, minmax(0, 1fr));
grid-cols-4	grid-template-columns: repeat(4, minmax(0, 1fr));
grid-cols-5	grid-template-columns: repeat(5, minmax(0, 1fr));
grid-cols-6	grid-template-columns: repeat(6, minmax(0, 1fr));
grid-cols-7	grid-template-columns: repeat(7, minmax(0, 1fr));
grid-cols-8	grid-template-columns: repeat(8, minmax(0, 1fr));
grid-cols-9	grid-template-columns: repeat(9, minmax(0, 1fr));
grid-cols-10	grid-template-columns: repeat(10, minmax(0, 1fr));
grid-cols-11	grid-template-columns: repeat(11, minmax(0, 1fr));
grid-cols-12	grid-template-columns: repeat(12, minmax(0, 1fr));

Class	Properties
grid-cols-none	grid-template-columns: none;

Especificando as colunas em uma grade

Use os `grid-cols-{n}` utilitários para criar grades com n colunas de tamanhos iguais.



```
<div class="grid grid-cols-4 gap-4">
  <div>01</div>
  <!-- ... -->
  <div>09</div>
</div>
```

Breakpoints and media queries

Você também pode usar modificadores de variantes para direcionar consultas de mídia, como pontos de interrupção responsivos, modo escuro, preferência por movimento reduzido e muito mais. Por exemplo, use `md:grid-cols-6` para aplicar o utilitário `grid-cols-6` apenas em tamanhos de tela médios e superiores.

```
<div class="grid grid-cols-1 md:grid-cols-6">
  <div>01</div>
  <!-- ... -->
  <div>06</div>
</div>
```

Início/Fim da Coluna da Grade

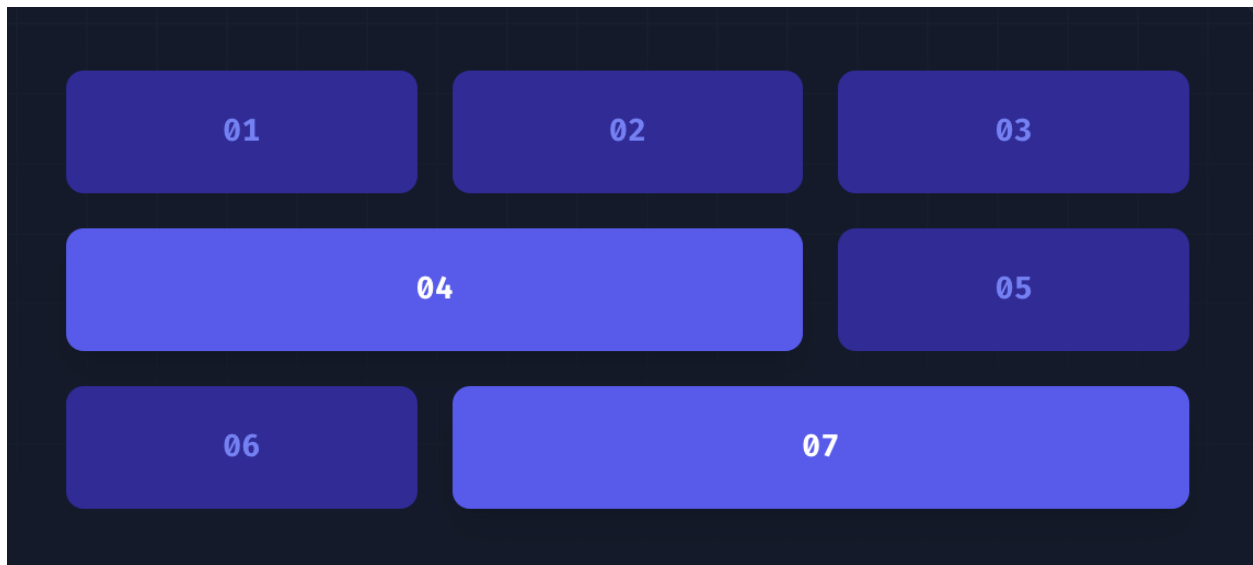
Utilitários para controlar como os elementos são dimensionados e colocados nas colunas da grade.

Class	Properties
col-auto	grid-column: auto;
col-span-1	grid-column: span 1 / span 1;
col-span-2	grid-column: span 2 / span 2;
col-span-3	grid-column: span 3 / span 3;
col-start-1	grid-column-start: 1;
col-start-2	grid-column-start: 2;
col-start-3	grid-column-start: 3;
col-end-1	grid-column-end: 1;
col-end-2	grid-column-end: 2;
col-end-3	grid-column-end: 3;

Colunas abrangentes

Use os utilitários `col-span-{n}` para fazer um elemento abranger n colunas.

```
<div class="grid grid-cols-3 gap-4">
  <div class="...">01</div>
  <div class="...">02</div>
  <div class="...">03</div>
  <div class="col-span-2 ...">04</div>
  <div class="...">05</div>
  <div class="...">06</div>
  <div class="col-span-2 ...">07</div>
</div>
```

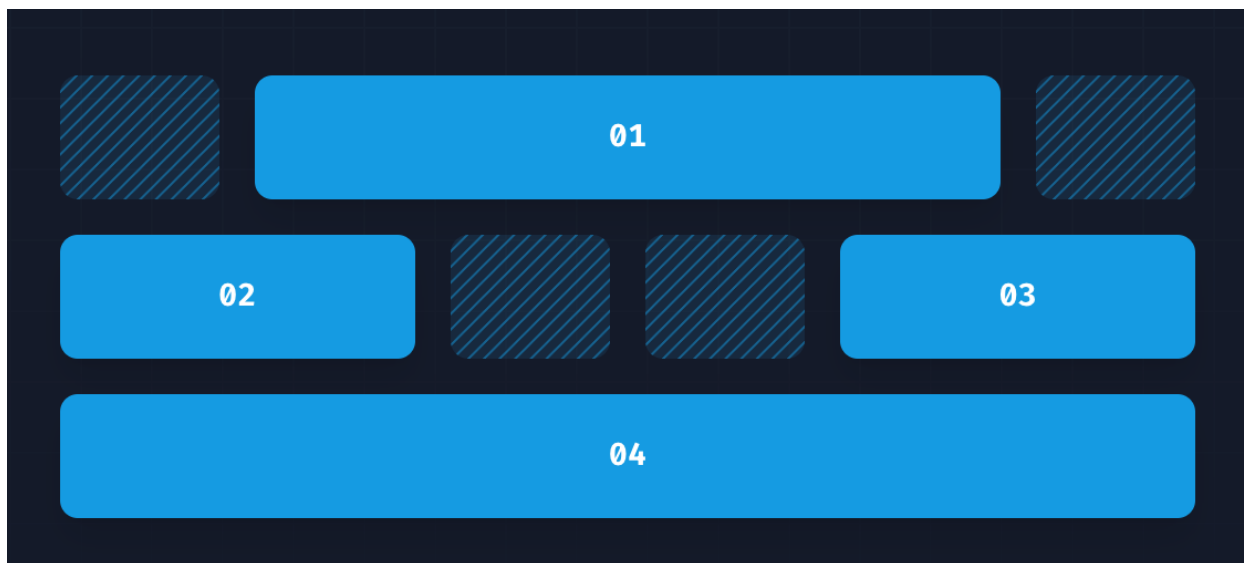


Linhas iniciais e finais

Use os utilitários `col-start-{n}` e `col-end-{n}` para fazer com que um elemento comece ou termine na *enésima* linha da grade. Eles também podem ser combinados com `col-span-{n}` para abranger um número específico de colunas.

Observe que as linhas de grade CSS **começam em 1**, não em 0, portanto, um elemento de largura total em uma grade de 6 colunas começaria na linha 1 e terminaria na linha 7.

```
<div class="grid grid-cols-6 gap-4">
  <div class="col-start-2 col-span-4 ...">01</div>
  <div class="col-start-1 col-end-3 ...">02</div>
  <div class="col-end-7 col-span-2 ...">03</div>
  <div class="col-start-1 col-end-7 ...">04</div>
</div>
```



Linhas do modelo de grade

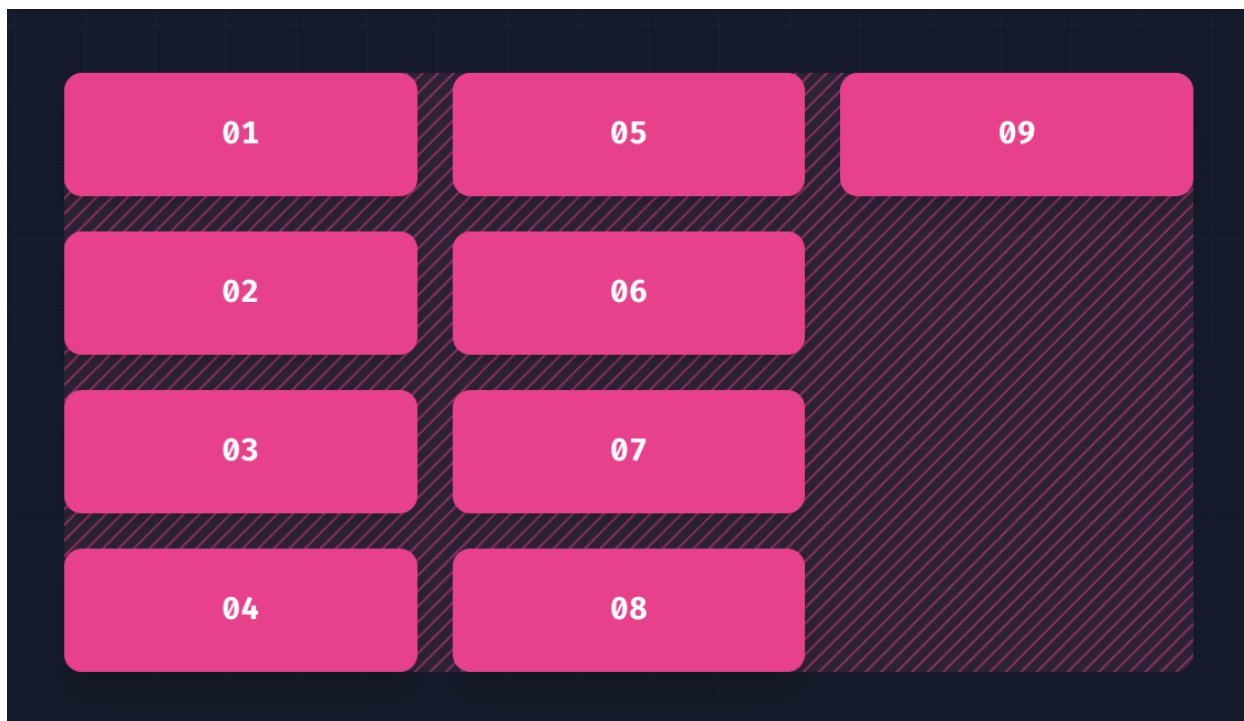
Utilitários para especificar as linhas em um layout de grade.

Class	Properties
grid-rows-1	grid-template-rows: repeat(1, minmax(0, 1fr));
grid-rows-2	grid-template-rows: repeat(2, minmax(0, 1fr));
grid-rows-3	grid-template-rows: repeat(3, minmax(0, 1fr));

Especificando as linhas em uma grade

Use os utilitários `grid-rows-{n}` para criar grades com n linhas de tamanhos iguais.

```
<div class="grid grid-rows-4 grid-flow-col gap-4">
  <div>01</div>
  <!-- ... -->
  <div>09</div>
</div>
```

Início/fim da linha da grade

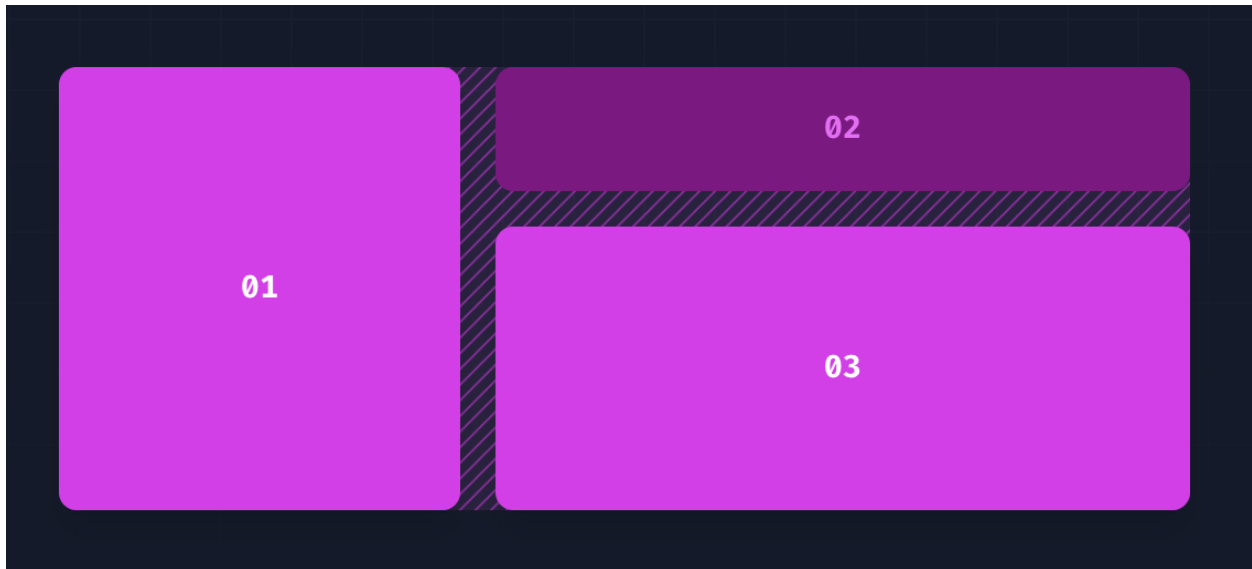
Utilitários para controlar como os elementos são dimensionados e colocados nas linhas da grade.

Classe	Propriedades
row-auto	grid-row: auto;
row-span-1	grid-row: span 1 / span 1;
row-span-2	grid-row: span 2 / span 2;
row-span-3	grid-row: span 3 / span 3;
row-span-4	grid-row: span 4 / span 4;
row-start-1	grid-row-start: 1;
row-start-2	grid-row-start: 2;
row-start-3	grid-row-start: 3;
row-end-1	grid-row-end: 1;
row-end-2	grid-row-end: 2;
row-end-3	grid-row-end: 3;

Linhas abrangentes

Use os utilitários `row-span-{n}` para fazer um elemento abranger n linhas.

```
<div class="grid grid-rows-3 grid-flow-col gap-4">
  <div class="row-span-3 ...">01</div>
  <div class="col-span-2 ...">02</div>
  <div class="row-span-2 col-span-2 ...">03</div>
</div>
```

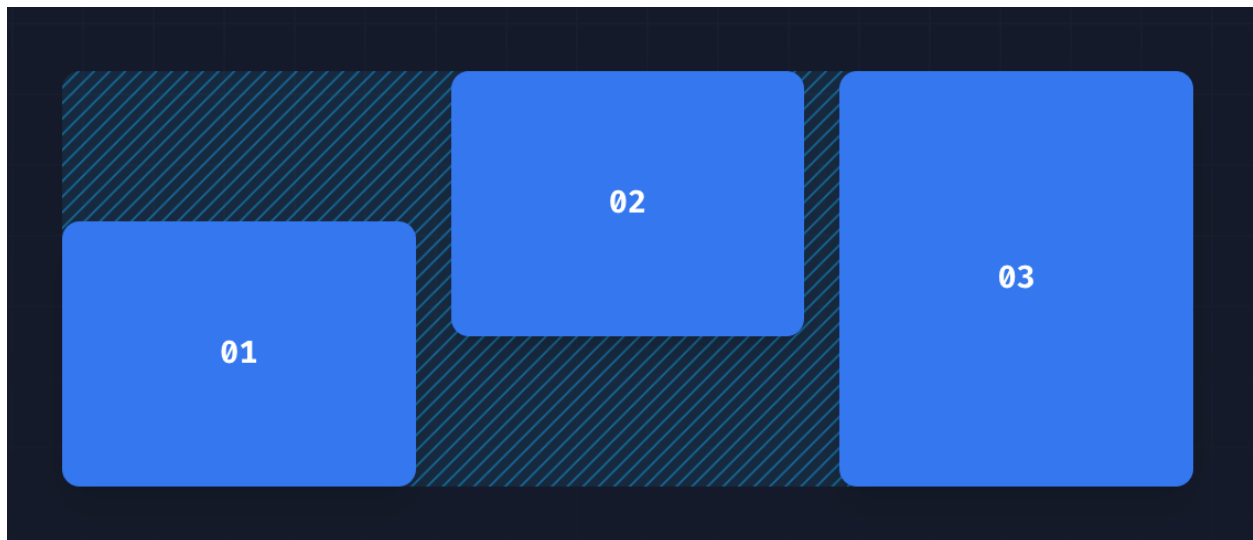


Linhas iniciais e finais

Use os utilitários `row-start-{n}` e `row-end-{n}` para fazer com que um elemento comece ou termine na *enésima* linha da grade. Eles também podem ser combinados com os utilitários `row-span-{n}` para abranger um número específico de linhas.

Observe que as linhas de grade CSS começam em 1, não em 0, portanto, um elemento de altura total em uma grade de 3 linhas começaria na linha 1 e terminaria na linha 4.

```
<div class="grid grid-rows-3 grid-flow-col gap-4">
  <div class="row-start-2 row-span-2 ...">01</div>
  <div class="row-end-3 row-span-2 ...">02</div>
  <div class="row-start-1 row-end-4 ...">03</div>
</div>
```



Fontes:

Installation - Tailwind CSS

The simplest and fastest way to get up and running with Tailwind CSS from scratch is with the Tailwind CLI tool.

 <https://tailwindcss.com/docs/installation>

 tailwindcss

Getting Started

Installation

The simplest and fastest way to get up and running with Tailwind CSS from scratch is with the Tailwind CLI tool.